Assignment 5

```c
#define MAX 10

// -------- FUNCTION DECLARATIONS --------
void addMatrix(int r, int c);
void saddlePoint(int r, int c);
void inverseMatrix();
void checkMagicSquare(int n);

// -------- MAIN --------
int main() {
    int choice, r, c, n;

    do {
        printf("\n--- MATRIX OPERATIONS MENU ---\n");
        printf("1. Addition of Matrices\n");
        printf("2. Saddle Point\n");
        printf("3. Inverse of Matrix (2x2)\n");
```

```c
#define MAX 10

// -------- FUNCTION DECLARATIONS --------
void addMatrix(int r, int c);
void saddlePoint(int r, int c);
void inverseMatrix();
void checkMagicSquare(int n);

// -------- MAIN --------
int main() {
    int choice, r, c, n;

    do {
        printf("\n--- MATRIX OPERATIONS MENU ---\n");
        printf("1. Addition of Matrices\n");
        printf("2. Saddle Point\n");
        printf("3. Inverse of Matrix (2x2)\n");
```

```
    case 3:
        inverseMatrix();
        break;

    case 4:
        printf("Enter size of square matrix: ");
        scanf("%d", &n);
        checkMagicSquare(n);
        break;

    case 5:
        printf("Exiting...\n");
        break;

    default:
        printf("Invalid choice!\n");
}
```

```
for(int i=0;i<r;i++)
    for(int j=0;j<c;j++)
        sum[i][j] = A[i][j] + B[i][j];

printf("Resultant Matrix:\n");
for(int i=0;i<r;i++) {
    for(int j=0;j<c;j++)
        printf("%d ", sum[i][j]);
    printf("\n");
}


-------- SADDLE POINT --------
d saddlePoint(int r, int c) {
int A[MAX][MAX];
int found = 0;
```

```c
printf("Enter matrix:\n");
for(int i=0;i<r;i++)
    for(int j=0;j<c;j++)
        scanf("%d",&A[i][j]);

for(int i=0;i<r;i++) {
    int min = A[i][0], col = 0;

    // find row minimum
    for(int j=1;j<c;j++) {
        if(A[i][j] < min) {
            min = A[i][j];
            col = j;
        }
    }

    // check if it's max in that column
    int k;
    for(k=0;k<r;k++) {
```

```
    for(k=0;k<r;k++) {
        if(A[k][col] > min)
            break;
    }

    if(k == r) {
        printf("Saddle Point: %d\n", min);
        found = 1;
    }
}

if(!found)
    printf("No Saddle Point found.\n");


-------- INVERSE (2x2) --------
d inverseMatrix() {
float A[2][2], det;
```

```
    for(k=0;k<r;k++) {
        if(A[k][col] > min)
            break;
    }

    if(k == r) {
        printf("Saddle Point: %d\n", min);
        found = 1;
    }
}

if(!found)
    printf("No Saddle Point found.\n");


-------- INVERSE (2x2) --------
d inverseMatrix() {
float A[2][2], det;
```

```c
// check columns
for(int j=0;j<n;j++) {
    int col_sum = 0;
    for(int i=0;i<n;i++)
        col_sum += A[i][j];
    if(col_sum != magic_sum) {
        printf("Not a Magic Square\n");
        return;
    }
}

// diagonals
for(int i=0;i<n;i++) {
    sum_diag1 += A[i][i];
    sum_diag2 += A[i][n-i-1];
}
```

```c
  if(sum_diag1 == magic_sum && sum_diag2 == magic_sum)
      printf("It is a Magic Square\n");
  else
      printf("Not a Magic Square\n");
}
```

output-

```
Enter your choice: 1
Enter rows and columns: 2 2
Enter Matrix A:
1 2
3 4
Enter Matrix B:
1 8
1 9
Resultant Matrix:
2 10
4 13

--- MATRIX OPERATIONS MENU
1. Addition of Matrices
2. Saddle Point
3. Inverse of Matrix (2x2)
4. Check Magic Square
5. Exit
Enter your choice: 3
```

```
Enter your choice: 3
Enter 2x2 matrix:
1 8
9 1
Inverse Matrix:
-0.01 0.11
0.13 -0.01

--- MATRIX OPERATIONS MENU ---
1. Addition of Matrices
2. Saddle Point
3. Inverse of Matrix (2x2)
4. Check Magic Square
5. Exit
Enter your choice: 4
Enter size of square matrix: 2 2
Enter matrix:
1 2
3 4
```

```
1 2
3 4
Not a Magic Square

--- MATRIX OPERATIONS MENU ---
1. Addition of Matrices
2. Saddle Point
3. Inverse of Matrix (2x2)
4. Check Magic Square
5. Exit
```